

DRAGONFLY V2 — LOGICIEL RASPBERRY PI

DOC-REF: DfV2-RPI5-SW

Guide d'Architecture, Installation, Services & Maintenance

1. PRÉSENTATION

Le Raspberry Pi 5 est utilisé dans le projet **DragonFly V2** principalement pour :

- Afficher l'interface utilisateur sur l'écran tactile ;
- Récupérer et afficher les informations de batterie ;
- Permettre l'arrêt logiciel du Raspberry Pi depuis l'interface ;
- Recevoir le flux Pixel Streaming provenant du PC hôte.

Une fonction d'estimation de proximité par microphone a également été développée sous forme de prototype, mais elle n'est pas encore intégrée au démarrage automatique du système.

2. ARCHITECTURE GÉNÉRALE

Le fonctionnement global est réparti entre trois équipements interconnectés :

ESP32

Récupère les commandes physiques :

- Volant / capteurs Hall
- Roller & joystick
- Boutons & tactiles

→ Transmet au PC en manette BLE HID.

PC Hôte

Exécute les composants centraux :

- CockpitCoach (UE5)
- Signalling WebServer
- Interface Web DragonFly

→ Diffuse l'IHM et le flux vidéo sur le LAN.

Raspberry Pi 5

Connecté au PC via le réseau local :

- Chromium Kiosque plein écran
- API Batterie Flask (INA219)
- Service Power-Off logiciel

→ Exécute les services Python locaux.

3. LANCEMENT DE L'INTERFACE

Le script principal est : `/home/yasmine/dragonfly-kiosk.sh`

Son rôle est d'exécuter la séquence d'initialisation suivante :

1. Attendre que le PC hôte soit accessible sur le réseau ;
2. Lancer Chromium ;
3. Ouvrir automatiquement l'interface DragonFly ;
4. Afficher Chromium en mode kiosque plein écran.

Le PC est identifié sur le réseau par son hostname (ex. `laptop-yasmine.local`), évitant ainsi la dépendance à une adresse IP fixe.

4. SURVEILLANCE DE LA BATTERIE

La surveillance est réalisée avec un capteur **INA219** connecté au Raspberry Pi par le bus I²C.

Répertoire racine : `/home/yasmine/Desktop/test-autonomie/`

- Serveur principal : `api-server.py`
- Module de lecture : `BattMonitor.py`
- Service systemd : `dragonfly-battery.service` (démarrage automatique)

Commandes de contrôle du service :

```
# Vérifier le statut
sudo systemctl status dragonfly-battery.service

# Redémarrer le service
sudo systemctl restart dragonfly-battery.service

# Tester la santé de l'API REST
curl http://127.0.0.1:5000/health

# Relever la télémétrie batterie
curl http://127.0.0.1:5000/battery
```

Note : Si l'INA219 n'est pas branché, une erreur I²C est retournée. Cela ne signifie pas que le serveur Flask est hors-service.

5. ARRÊT LOGICIEL DU RASPBERRY PI

Un serveur Python local écoute les requêtes d'arrêt envoyées depuis l'IHM tactile.

- Script dédié : `/home/yasmine/Desktop/power-server.py`
- Service systemd : `dragonfly-power.service` (démarrage automatique)

```
# Vérifier l'état du service d'extinction
sudo systemctl status dragonfly-power.service

# Redémarrer le service
sudo systemctl restart dragonfly-power.service
```

6. COMMUNICATION AVEC LE PC

Le Raspberry Pi et le PC doivent impérativement résider sur le même sous-réseau local.

```
# Contrôler la connexion Wi-Fi
nmcli device status

# Identifier le SSID actif
iwgetid

# Afficher l'adresse IP du Raspberry Pi
hostname -I

# Tester la résolution mDNS / Ping avec le PC
ping laptop-yasmine.local

# Tester l'accessibilité du serveur Web sur le port 80
nc -vz laptop-yasmine.local 80
```

Ports réseaux utilisés par Pixel Streaming :

- **Port 80** : Player / Interface Web utilisateur ;
- **Port 8888** : Streamer Unreal Engine ;
- **Port 8889** : Serveur SFU.

7. PROTOTYPE DE DÉTECTION DE PROXIMITÉ PAR MICROPHONE

Une fonction expérimentale à base d'analyse acoustique permet d'estimer la proximité d'obstacles. Ce module n'est pas encore intégré au démarrage automatique.

- Script principal : `test-plot-with-api.py`
- Script de test historique : `test-plot.py`
- Interface radar : `6_Distance.html`

Microphone → Acquisition Audio (48 kHz) → Fenêtre Hanning → FFT → Analyse bande 11-14 kHz → Calcul Énergie → Lissage → WebSocket → Radar IHM

⚠ IMPORTANT : La conversion de l'énergie acoustique en distance métrique (cm) n'est pas calibrée. Cette fonction est un prototype de proximité et non un télémètre certifié.

8. SERVICES DRAGONFLY

Vérification rapide de la pile de services système :

```
# Lister les services DragonFly actifs
systemctl --type=service --state=running | grep dragonfly

# Inspecter le fichier de configuration des services
sudo systemctl cat dragonfly-battery.service
sudo systemctl cat dragonfly-power.service
```

9. ORGANISATION CONSEILLÉE POUR LA SAUVEGARDE

```
Raspberry_Pi/
├─ kiosk/
│  └─ dragonfly-kiosk.sh
├─ battery/
│  ├─ api-server.py
│  ├─ BattMonitor.py
│  ├─ battery_state.json
│  └─ requirements.txt
├─ power/
│  ├─ power-server.py
│  └─ requirements.txt
├─ services/
│  ├─ dragonfly-battery.service
│  └─ dragonfly-power.service
└─ prototypes/
   └─ distance/
      ├─ test-plot-with-api.py
      ├─ test-plot.py
      └─ 6_Distance.html
```

10. ÉTAT ACTUEL DU SYSTÈME

FONCTIONNALITÉ	ÉTAT / STATUT
Interface Chromium kiosk	✓ Fonctionnelle
Communication Raspberry ↔ PC	✓ Fonctionnelle
Pixel Streaming	✓ Fonctionnel
API Batterie (Flask)	✓ Fonctionnelle
Lecture INA219	⚠ À utiliser avec capteur connecté
Arrêt logiciel Raspberry	✓ Fonctionnel
Autostart Batterie & Power	✓ Fonctionnels
Détection acoustique / Radar	🔧 Prototype
Conversion Énergie → Distance	⚙️ À calibrer

11. PROCÉDURE DE REPRISE DU PROJET

1. Vérifier la connectivité réseau du Raspberry Pi ;
2. S'assurer que le PC hôte est allumé et joignable ;
3. Contrôler les services `dragonfly-battery` et `dragonfly-power` ;
4. Vérifier l'exécution du script `dragonfly-kiosk.sh` ;
5. Lancer l'environnement Pixel Streaming côté PC ;
6. Valider la présence du flux sur l'écran du Raspberry ;
7. Brancher physiquement l'INA219 avant d'exécuter la mesure batterie.